



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026-27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.:12

Team No.:2

Project Title:Argument Passing

Team Members

Roll Number	Student Name	Signature
2520090205	A.JhansiPriya	Jhansi Priya
2520090177	Keerthi Priya Ande	Keerthi Priya

1.Abstract

Argument passing is an important concept in Operating Systems and Systems Programming that allows a program to receive information when it is executed. This project focuses on demonstrating how command-line arguments are passed from the Linux shell to a C program and how these arguments are received and processed using argc and argv. The project provides a practical understanding of program execution and the role of the operating system in supplying input to a running program.

The proposed solution will be implemented in a Linux/Ubuntu environment using C. The program will accept different numbers of command-line arguments, identify the number of arguments received, and display their corresponding values. To demonstrate the Operating Systems aspect of the project, process-related Linux/POSIX mechanisms such as fork(), exec(), and wait() will be used to show how a program can be executed with specified arguments.

The implementation will be tested with different test cases, including no arguments, a single argument, and multiple arguments. The project will include source code, build instructions, test results, screenshots, and a working demonstration.

The expected outcome is a clear practical understanding of argument passing, process execution, command-line interfaces, and Linux/POSIX system programming mechanisms.

2. Problem Statement

The project aims to demonstrate and understand how arguments are passed to a program during its execution in a Linux environment. Programs often require input values when they are started, and understanding how these values are received and processed is an important Systems Programming concept.

The project focuses on command-line argument passing in C using `argc` and `argv`. It demonstrates how the Linux shell provides arguments to a program and how the program accesses and processes those arguments. The project also relates argument passing to process execution by using Linux/POSIX mechanisms such as `fork()` and `exec()`.

This project is relevant to Operating Systems and Systems Programming because it provides practical understanding of process creation, program execution, command-line interfaces, and the mechanisms used to pass information to an executing program. The implementation is focused on Linux/POSIX system programming concepts rather than generic application development.

3. Objectives

1. To understand the concept of argument passing and how command-line arguments are received by a program in Linux.
2. To implement a C program that accepts and processes command-line arguments using `argc` and `argv`.
3. To demonstrate argument passing during process execution using Linux/POSIX system calls such as `fork()`, `exec()`, and `wait()`.
4. To test the argument-passing implementation with different inputs and verify the correctness of the received arguments and program output.

4. Proposed Methodology

The project will be designed and implemented in a Linux/Ubuntu environment using the C programming language. The proposed solution will demonstrate how command-line arguments are passed from the Linux shell to a program and how the program receives and processes them using `argc` and `argv`.

The source code will first be written and compiled using the GCC compiler. The program will then be executed from the Linux terminal by providing different command-line arguments. The number of arguments and their corresponding values will be identified and displayed by the program.

The project will also demonstrate the process of executing a program with arguments using Linux/POSIX mechanisms. Where required, `fork()` will be used to create a child process, `exec()` will be used to execute the program with arguments, and `wait()` will be used by the parent process to wait for the child process.

Finally, different test cases will be performed using no arguments, single arguments, and multiple arguments. The outputs will be verified and documented using terminal results and screenshots.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
Command-Line Arguments	To provide input values to the program through the Linux terminal.
argc	To determine the number of command-line arguments received by the program.
argv	To access and process the actual command-line argument values.
Process Creation	To demonstrate how a new process is created for program execution.
fork()	To create a child process from the parent process.
exec()	To execute a program with the specified command-line arguments.
wait()	To allow the parent process to wait for the child process to complete.
Process ID (PID)	To identify and distinguish the parent and child processes.
Linux Shell	To execute the program and pass arguments through the command line.
POSIX APIs	To provide standard interfaces for process-related system programming.
Process Synchronization	To coordinate the execution of the parent and child processes.
Standard Input/Output	To display the received arguments and execution results in the terminal.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520090205	A.Jhansi Priya	Design and implementation of the argument-passing program using argc and argv.
2520090177	Keerthi Priya Ande	Implementation and testing of process-based argument passing using fork(), exec(), and wait().

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Operating system environment for developing, compiling, and executing the project.
C / C++	Programming language used to implement argument passing and process-related concepts.
GCC Compiler	Used to compile the C source code and create the executable file.
Linux Terminal	Used to execute the program and provide command-line arguments.

Make / Makefile	
POSIX Libraries / APIs	Used to automate the compilation and build process.
VS Code / Text Editor	Used to access process-related functions such as fork(), exec(), and wait().
Git / GitHub	Used to write and edit the C source code and project files.
	Used to maintain, manage, and submit the project source code and documentation.

8. Expected Outcome

The project is expected to successfully demonstrate the working of argument passing in a Linux environment. The implemented C program will accept command-line arguments from the Linux terminal and correctly identify and process them using argc and argv.

The project will demonstrate how arguments are passed during program execution and how Linux/POSIX process mechanisms such as fork(), exec(), and wait() can be used in the argument-passing process. The program will display the number of arguments received and their corresponding values.

Different test cases will be performed using no arguments, a single argument, and multiple arguments to verify the correctness of the implementation. The final output will include successful program execution, terminal results, screenshots, test-case evidence, and a working demonstration.

The expected outcome is a clear practical understanding of command-line argument passing, process creation, program execution, process synchronization, and Linux/POSIX system programming concepts.

9. GitHub Repository

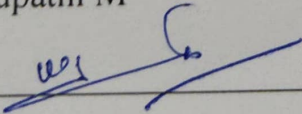
GitHub Repository Link: <https://github.com/keerthipriya2008/klh-csit-2025-2029-02-ArgumentPassing>

Repository Created: yes

☒ Yes ☐ No

Faculty Name and Signature

Faculty Name: Raghupathi M

Signature: 

Remarks: _____

Faculty Use

Project Approved: _____

☒ Yes ☐ No

Date: 31/08/2026

Faculty Signature: 